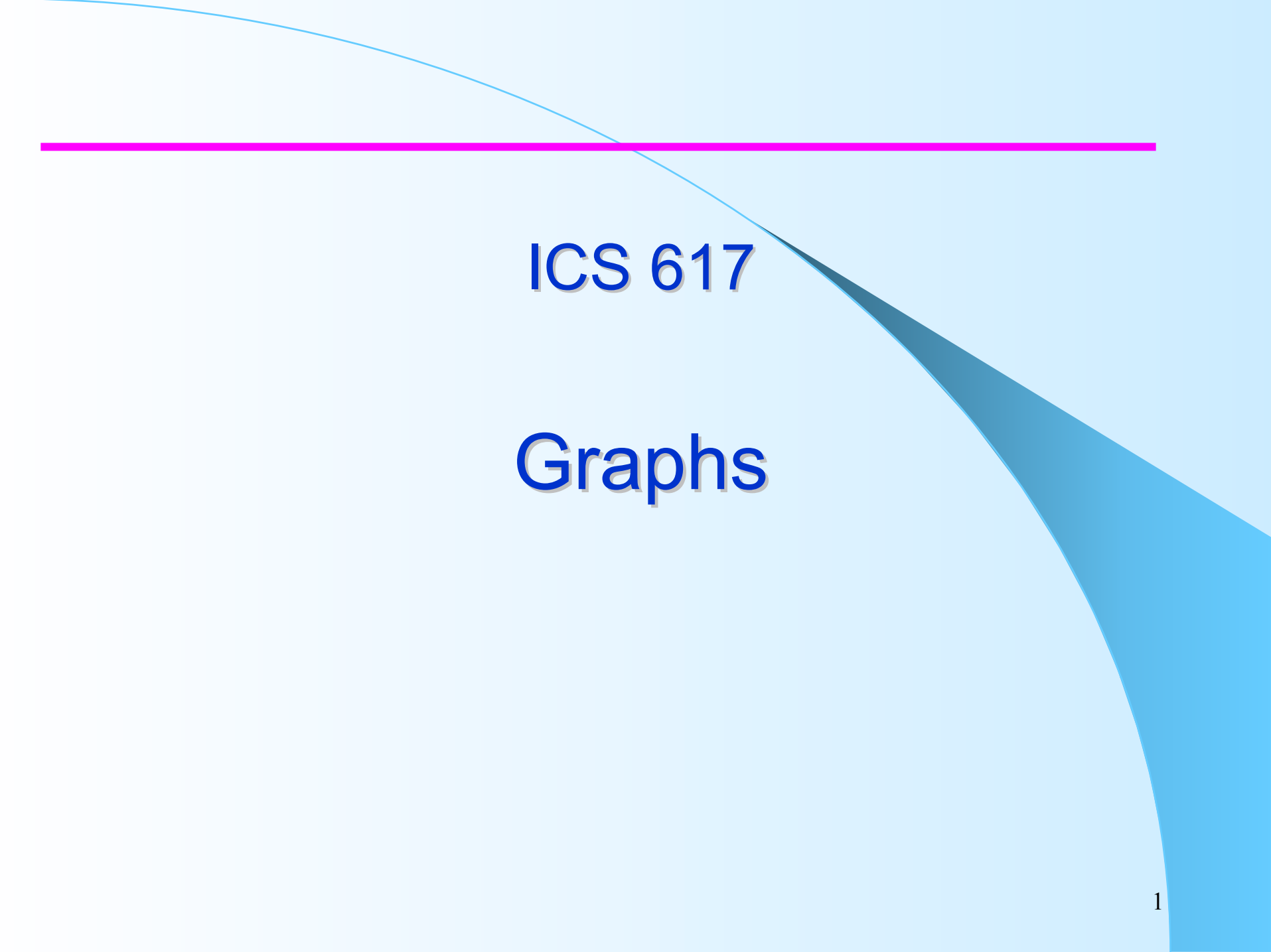




ICS 617

Graphs

Minimum Spanning Tree

- ❑ Creating a minimum spanning tree is more complicated with weighted graphs than with un-weighted ones
- ❑ This is because we have to put the weight of the edges into consideration (hence the minimum cost spanning tree)
- ❑ Two algorithms are usually used to construct minimum spanning trees
- ❑ These are Prim's algorithm and Kruskal's algorithm
- ❑ We look at Prim's algorithm

Prim's Algorithm

- ❑ We need to construct a minimum spanning tree for $G=(V,E)$
- ❑ Initially, Prim's algorithm has only one node in the spanning tree
- ❑ The algorithm then adds nodes to the spanning tree, one node at a time based on the edge costs to connect to the nodes already in the tree
- ✓ The algorithm considers edges that connect nodes in the spanning tree (**S**) to nodes not in the spanning tree **V-S**
- ✓ Among these edges, it adds the least cost edge (**u,v**) into the spanning tree,
- ✓ **u** is a node in the **S** while **v** is in **V-S**
- ✓ When (**u,v**) is added into the spanning tree, **v** is also added into the tree
- ✓ This doesn't create a cycle because this new node doesn't provide a new path to reach any other node in the spanning tree except the one it was adjacent to in the edge just added

Prim's Algorithm

- ❑ We need to store the connections/edges from the nodes in the spanning tree to the nodes not in the spanning tree (lets call these edges λ)
- ❑ Remember we will choose the minimum edge (u,v) from λ
- ❑ We need to store the edges in λ so that we can choose the minimum edge efficiently
- ❑ To achieve this aim, we use a priority queue
- ❑ Hence the minimum connecting edge (u,v) will be at the front of the priority queue
- ❑ This edge is removed from the queue and added to the minimum spanning tree

Prim's Algorithm

- Since λ contains edges that connects nodes in the spanning tree to nodes not in the spanning tree, once v is added to the tree, it is removed from $V-S$ and added to S
- If there was any other edge in λ connecting a node in the spanning tree to v it now needs to be removed from λ hence from the priority queue
- When we no longer have any nodes in $V-S$, then we will have finished constructing a minimum spanning tree for the $G=(V,E)$

Prim's Algorithm

- ❑ The performance of the Prim's algorithm depends on how the priority queue is implemented
- ❑ Queue operations like additions and deletions will be $O(n)$ if a list is used
- ❑ To improve performance a heap can be used to implement the priority queue

Prim's Algorithm (simplified)

```
PrimMCST( set &T)
{
    set U /*vertices in spanning tree*/;
    /* T = set of edges in spanning tree */
    T =  $\phi$ ;
    U = {0};
    while (U <> V) {
        /* Add least cost edge that adds node not in U */

        (u,v) = lowest cost edge st. u in U, v in V-U

        /* Add edge to spanning tree */
        Insert(T,u,v);
        /* Add v to set of connected vertices */
        Insert(U,v);
    } //end while
    Print(T);
} /*end PrimMCST */
```

```

PrimMCST(G(V,E))
{
/* C[N][N] adjacency cost matrix for G. It should
be initialised*/
set <edge> T;
set <vertex> U;
int lowcost[N], closest[N];
int i,j,k,index
/* T =set of edges in the spanning tree*/
/* U =set of nodes in the spanning tree*/
insert(U,0) //add node 0 into the spanning tree
/*if a node x is in the spanning tree then
closest[x] is set to -1 */
closest[0]=-1;

for(i=1;i<N; i++)
{
    /*store the cost of the edge from any node to
node 0 in the in the lowcost array*/
    lowcost[i]=C[0][i];
    /*Set node 0 as the node closest to the nodes
not in the spanning tree based on edge cost*/
    closest[i]=[0];
}

while(U<>V)
{
    /*select the node with the lowest index node
not in the spanning tree*/
    for(index=0;index<N;index++)
    {
        if(closest[index]<>-1)
        {
            break;
        } //end if
    } //end for
    //get the least cost edge
    min=lowcost[index];
}

```

```

for(i=index;i<N;i++)
{
    if(closest[i]<>-1 && leastcost[i]<min)
    {
        min=leastcost[i];
        k=i;
    } //end if
} //end for
/*add the node k into the set of nodes in the
spanning tree - U)*/
insert(U,k);
/*insert the associated edge into the spanning
tree*/
insert(T,closest[k],k);
/*set k as having been inserted into the
spanning tree*/
closest[k]=-1;
/*adjust the lowcosts taking into account the node
k that has been added into the spanning tree*/
for(j=0;j<N;j++)
{
    if(closest[i]<>-1 && C[k][j]<lowcost[j])
    {
        lowcost[j]= C[k][j];
        closest[j]=k;
    } //end if
} //end for

} //end while
} //end function

```

Prim's Algorithm Explanations

- ❑ Note that we need to maintain two arrays, these are $Lowcost[v]$ and $closest[v]$
- ❑ For a node v not in the spanning tree, $Closest[v]$ is the node u in the spanning tree that is closest (has the least cost) to the node v
- ❑ $Lowcost[v]$ is the cost of the edge (u, v)
 - ✓ This cost can be set to infinity if both u and v are already in the spanning tree

Example Graph

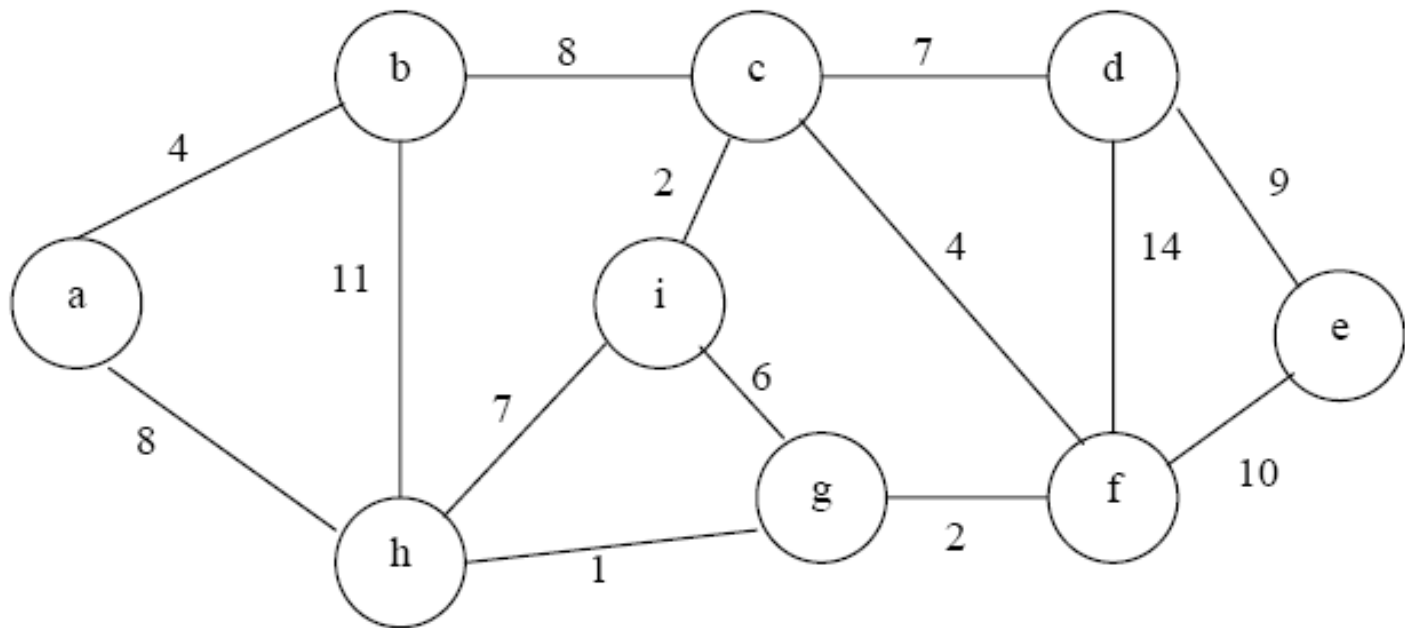
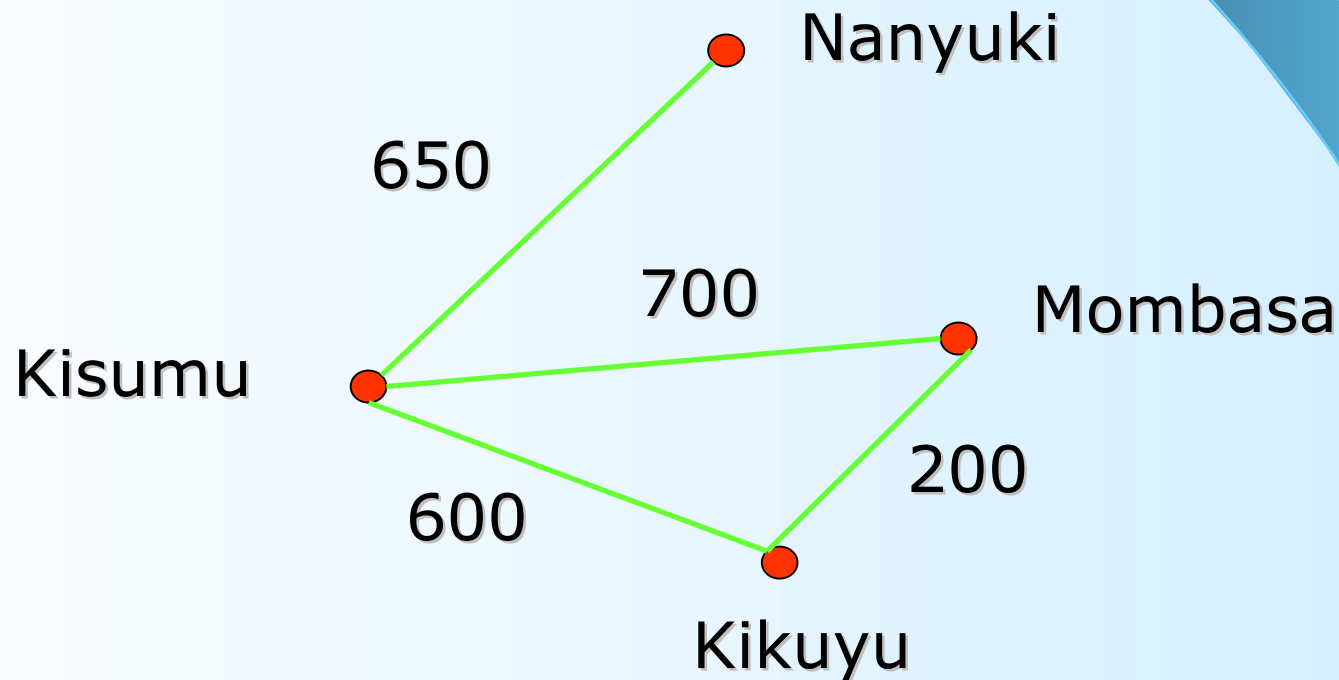


Fig 1: connected Graph

Shortest Path Problems

- We can assign weights to the edges of graphs, for example to represent the distance between cities in a railway network:



Shortest Path Problems

- ❑ Such weighted graphs can also be used to model computer networks with response times or costs as weights.
- ❑ One of the most interesting questions that we can investigate with such graphs is:
- ❑ What is the **shortest path** between two vertices in the graph, that is, the path with the **minimal sum of weights** along the way?
- ❑ This corresponds to the shortest train connection or the fastest connection in a computer network.

Dijkstra's Algorithm

- ❑ Dijkstra's algorithm is an iterative procedure that finds the shortest path between a node known as the source and any other node in the graph
- ❑ It proceeds by finding the length of the shortest path from the source to successive vertices and adding these vertices to a distinguished set of vertices S .
- ❑ For the vertices in S , we already know the shortest path from the source
- ❑ The set $V-S$ contains the nodes in V that have not been added to S .
- ❑ We could find the shortest path from the source node to a destination node but this takes about the same time as finding the shortest path to all other nodes
- ✓ The problem solved is hence also referred to as single source shortest paths

Dijkstra's Algorithm

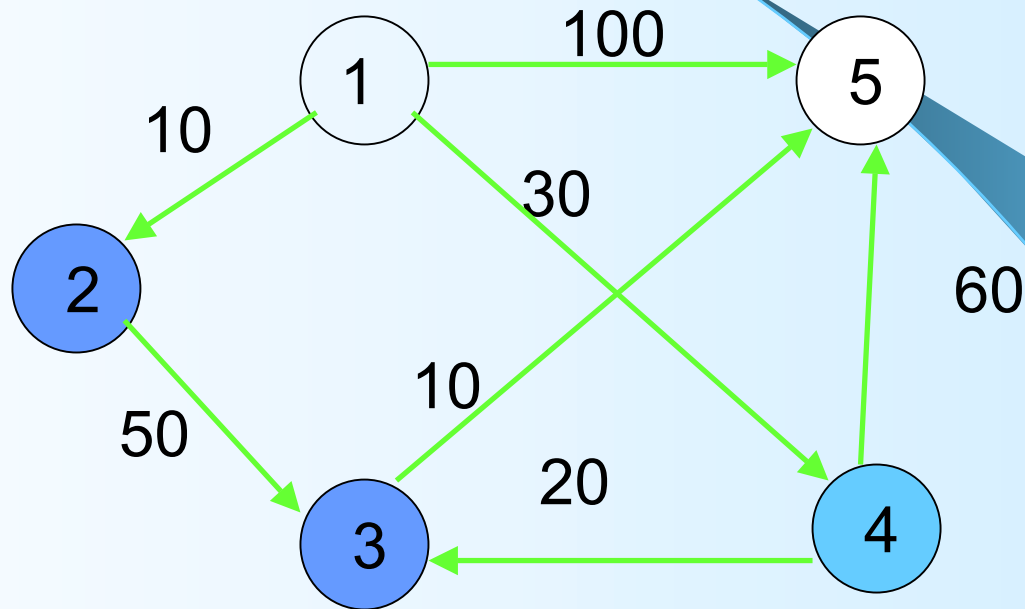
- ❑ The algorithm assumes we are given a directed graph $G=(V,E)$ where $V=\{1,2,\dots,n\}$ and vertex 1 is the source. C is a 2 dimensional array of costs, where $C[i,j]$ is the cost of moving from vertex i to j on arc $i \rightarrow j$. If arc $i \rightarrow j$ doesn't exist, assume $C[i,j] = \infty$ i.e its equal to infinity.
- ❑ $D[i]$ contains the cost of the current shortest path to vertex i .
- ❑ Using Dijkstra's algorithm, to get the path to a node, we have to have another array P of vertices such that $P[v]$ contains the vertex immediately before v in the shortest path
- ❑ Initialize $P[v]$ to 1 for all $v \neq 1$. Every time in the algorithm and $D[v] < D[w] + C[w,v]$ (Temp) then we set

$$P[v] := w$$

Dijkstra's Algorithm

```
function Dijkstra {
1. S:={1};
2. for i:=2 to n
3. D[i]:=C[1,i] //initialize D
4. For i:=1 to n-2 {
6.     choose a vertex w in V-S such that D[w] is a
        minimum
7.     add w to S
8.     for each vertex v in V-S {
9.         Temp:=D[w]+C[w,v]
10.        If (Temp<D[v]){
11.            D[v]:=Temp
12.            P[v]:=w
13.        }//end if
14.    }//end for
15. }//end for
16. }//end function
```

Dijkstra's Algorithm



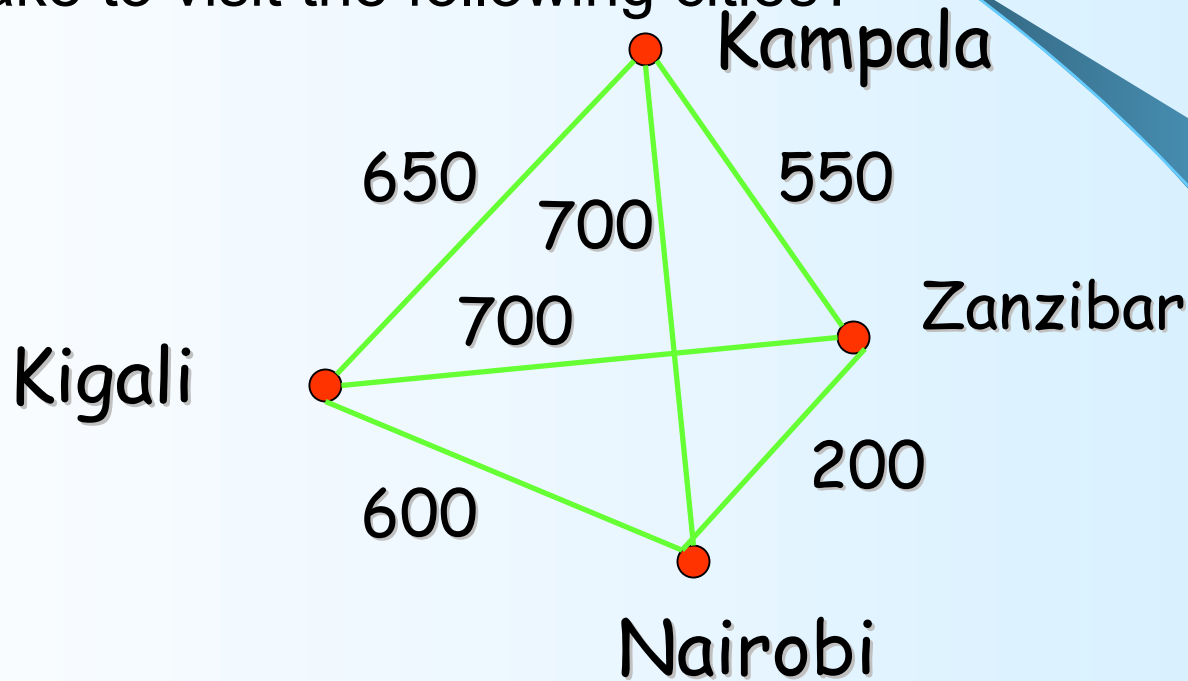
Given the above graph, use Dijkstra's algorithm to get the path to every other node from node 1?

The Traveling Salesman Problem

- ❑ The **traveling salesman problem** is one of the classical problems in computer science.
- ❑ A traveling salesman wants to visit a number of cities and then return to his starting point. Of course he wants to save time and energy, so he wants to determine the **shortest path** for his trip.
- ❑ We can represent the cities and the distances between them by a weighted, complete, undirected graph.
- ❑ The problem then is to find the **circuit of minimum total weight that visits each vertex exactly once**.

The Traveling Salesman Problem

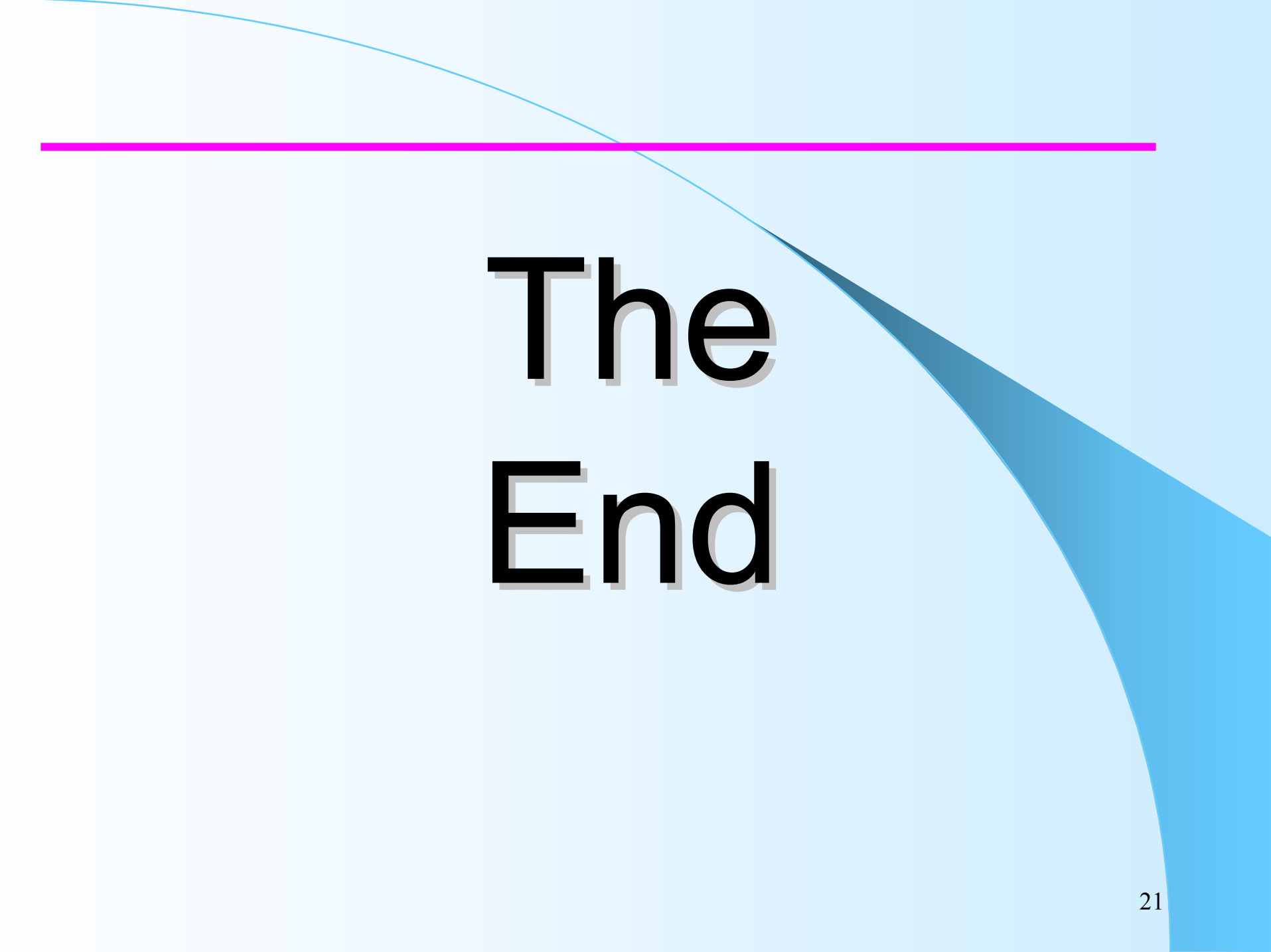
- ❑ **Example:** What path would the traveling salesman take to visit the following cities?



- ❑ **Solution:** The shortest path is Zanzibar, Nairobi, Kigali, Kampala, Zanzibar (2,000 miles).

The Traveling Salesman Problem

- ❑ Unfortunately, no algorithm solving the traveling salesman problem acceptable time complexity has been devised yet.
- ❑ This means that for large numbers of vertices, solving the traveling salesman problem is impractical.
- ❑ In these cases, we can use efficient **approximation algorithms** that determine a path whose length may be slightly larger than the traveling salesman's path.



The
End